

This PDF describes the expected outputs to be found at each step of GenAnT.

GenAnT output	1
Step 0: Preprocessing	3
Step 1: Repeat annotation and masking	4
Step 2: Generating protein-coding gene models	7
Stringtie	7
Braker	8
LiftOff	9
TOGA	10
Step 3: Combining and filtering gene models	13
Prepare transcripts	13
Evaluate transcripts	15
Transdecoder	15
Portcullis and regtools	15
Blastp	16
Transcript selection.	17
Step 4: Annotating non-coding RNA genes	18
MiRmachine	18
Seeding and evaluation	19
Incorporation into the GFF annotation file	19
Step 5: Sequence-similarity-based transfer of gene symbols	21
Orthofinder	21
Compiling gene symbols	22

A typical run of GenAnT will have several directories containing individual annotation tools.

full_annotation.geneSymbols.gff	mirmachine	braker_lr	stringtie_out
gene_symbol	assembly	toga_out1	RNAseq_alignment
ncRNA_analysis	transcript_selection	cactus_alnr1	ISOseq_alignment
orthofinder	braker_sr	liftoff	

GenAnT output

The final file is the “full_annotation.geneSymbols.gff” file, which contains annotated coding and non-coding genes.

In our snakemake pipeline, the final file is called with the “all” rule, and is found in the directory where you’ve directed your output annotations.:

```
...
rule all:
  input:
    config["outDir"]+"/full_annotation.geneSymbols.gff"
...
```

Below is an example of the “mikado.chr1G11” or “Rsph4a” gene model

```
chr1 Mikado_loci gene 1013193 1032813 20 + . ID=mikado.chr1G11;Name=Rsph4a;liftoff_gene=gene:ENSHGLG000000002525;toga_gene=Rsph4a;togar1_gene=Rsph4a;orthofinder_gene=Rsph4a;hierarchical_gene=Rsph4a
chr1 Mikado_loci mRNA 1013193 1032813 20 + . ID=mikado.chr1G11.1;Name=Rsph4a;Parent=mikado.chr1G11;alias=liftoff_transcript:ENSHGLT0000003467;primary=True;liftoff_gene=gene:ENSHGLG000000002525;toga_gene=Rsph4a;togar1_gene=Rsph4a;orthofinder_gene=Rsph4a;hierarchical_gene=Rsph4a
chr1 Mikado_loci exon 1013193 1013268 . + . ID=mikado.chr1G11.1.exon1;Parent=mikado.chr1G11.1;
chr1 Mikado_loci five_prime_UTR 1013193 1013268 . + . ID=mikado.chr1G11.1.five_prime_UTR1;Parent=mikado.chr1G11.1;
chr1 Mikado_loci exon 1014146 1014946 . + . ID=mikado.chr1G11.1.exon2;Parent=mikado.chr1G11.1;
chr1 Mikado_loci five_prime_UTR 1014146 1014326 . + . ID=mikado.chr1G11.1.five_prime_UTR2;Parent=mikado.chr1G11.1;
chr1 Mikado_loci CDS 1014327 1014946 . + 0 ID=mikado.chr1G11.1.CDS1;Parent=mikado.chr1G11.1;
chr1 Mikado_loci exon 1019616 1019850 . + . ID=mikado.chr1G11.1.exon3;Parent=mikado.chr1G11.1;
chr1 Mikado_loci CDS 1019616 1019850 . + 1 ID=mikado.chr1G11.1.CDS2;Parent=mikado.chr1G11.1;
chr1 Mikado_loci exon 1027274 1028017 . + . ID=mikado.chr1G11.1.exon4;Parent=mikado.chr1G11.1;
chr1 Mikado_loci CDS 1027274 1028017 . + 0 ID=mikado.chr1G11.1.CDS3;Parent=mikado.chr1G11.1;
chr1 Mikado_loci CDS 1029089 1029221 . + 0 ID=mikado.chr1G11.1.CDS4;Parent=mikado.chr1G11.1;
chr1 Mikado_loci exon 1029089 1029221 . + . ID=mikado.chr1G11.1.exon5;Parent=mikado.chr1G11.1;
chr1 Mikado_loci CDS 1029876 1029993 . + 2 ID=mikado.chr1G11.1.CDS5;Parent=mikado.chr1G11.1;
chr1 Mikado_loci exon 1029876 1029993 . + . ID=mikado.chr1G11.1.exon6;Parent=mikado.chr1G11.1;
chr1 Mikado_loci CDS 1031581 1031812 . + 1 ID=mikado.chr1G11.1.CDS6;Parent=mikado.chr1G11.1;
chr1 Mikado_loci exon 1031581 1032813 . + . ID=mikado.chr1G11.1.exon7;Parent=mikado.chr1G11.1;
chr1 Mikado_loci three_prime_UTR 1031813 1032813 . + . ID=mikado.chr1G11.1.three_prime_UTR1;Parent=mikado.chr1G11.1;
```

Our result for protein-coding genes has a standard GFF format (i.e., found in NCBI) with a few additions. Specifically, we include the gene symbols from each annotation source in the “gene” and “mRNA” regions. The “Name” is the consensus gene symbol. Liftoff_gene, togar1_gene, and “orthofinder_gene” represent the gene symbol called in each method. The “alias” element in the “mRNA” rows represents the annotation tool that had the highest-scoring gene model (from Mikado) for that transcript, and therefore provided the “baseline” for the consensus model in the GFF.

The GFF file also contains non-coding gene models, which are subcategorized into the “ncRNA” type from Mikado, the “cmscan” type from Infernal, and miRNAs from mirmachine.

An example of a Mikado ncRNA annotation:

```
chr1 Mikado_loci ncRNA_gene 69710 70061 0 . . ID=mikado.chr1G3;
chr1 Mikado_loci ncRNA 69710 70061 0 . . ID=mikado.chr1G3.1;Parent=mikado.chr1G3;alias=stringtie_MSTRG.2.1;primary=True;
chr1 Mikado_loci exon 69710 70061 . . . ID=mikado.chr1G3.1.exon1;Parent=mikado.chr1G3.1;
```

An example of a cmscan rRNA annotation.

```
chr1 cmscan rRNA 7988223 7989384 594.7 - . ID=SSU_rRNA_eukarya.1;Parent=SSU_rRNA_eukarya;evalue=1.1e-179;gbkey=ncRNA;RFamID=RF01960;product=Eukaryotic small subunit ribosomal RNA;
chr1 cmscan ncRNA_gene 7988223 7989384 594.7 - . ID=SSU_rRNA_eukarya;evalue=1.1e-179;RFamID=RF01960;product=Eukaryotic small subunit ribosomal RNA;gene_biotype=rRNA;
chr1 cmscan exon 7988223 7989006 64.9 - . ID=SSU_rRNA_bacteria.1.exon1;Parent=SSU_rRNA_bacteria.1;evalue=7.9e-19;RFamID=RF00177;product=Bacterial small subunit ribosomal RNA;
```

An example of a microRNA annotation:

```
ptg0000011 MirMachine ncRNA_gene 5238261 5238322 88.1 - . ID=Mir-652;evalue=1.4e-16;gene_biotype=microRNA;
ptg0000011 MirMachine microRNA 5238261 5238322 88.1 - . ID=Mir-652.1;Parent=Mir-652;evalue=1.4e-16;gbkey=ncRNA;
```


Step 0: Preprocessing

Before starting any of the “steps” described in our tutorial, we build the initial directory structure and copy the genome assembly, RNA-seq, and ISO-seq data in.

The “preprocess_data” rule generates the “assembly” directory, copies your assembly fasta file into it, and generates the “transcript_selection” directory. The “assembly” directory contains the unmasked (and eventually masked) FASTA file used in each step of assembly. The “transcript_selection” directory contains the output of each annotation GFF file as well as the integration of gene models generated by Mikado. We will elaborate further below.

The RNAseq_alignment and ISOseq_alignment copy the directories containing your bam files into GenAnT. We copy these files instead of linking them because `Singularity –bind` sometimes has issues with linked files.

```
...
rule preprocess_data:
  input:
    config["assemblyFile"]
  output:
    config["outDir"]+"/assembly/assembly.fa"
...
...
rule copy_RNAseq:
  input:
    config["rnaseqDir"]
  output:
    directory(config["outDir"]+"/RNAseq_alignment")
...
...
rule copy_ISOseq:
  input:
    config["isoseqDir"]
  output:
    directory(config["outDir"]+"/ISOseq_alignment")
..
```

Step 1: Repeat annotation and masking

We use Earl Grey to build a repeat library, mask, and annotate repeats. These results are stored in the “earl_grey” directory. If you input a softmasked assembly, this directory is not created.

In the example data, we are annotating a chromosome from the naked mole-rat (*Heterocephalus glaber*). The output of EarlGrey is in “earl_grey/heterocephalus_glaber_EarlGrey”, but it will change to whatever species you select. Repeat masking is completed with the “mask_assembly” rule.

```
...
rule mask_assembly:
  input:
    config["outDir"]+"/assembly/assembly.fa"
  output:
    config["outDir"]+"/assembly/assembly.softmasked.fa"
...
```

Earl Grey is itself a wrapper of many repeat annotation tools, resulting in many directories.

The EarlGrey output directory

```
0 Mar 12 00:05 heterocephalus_glaber_heliano
198 Mar 12 00:26 heterocephalus_glaber_RepeatMasker
47 Mar 12 00:26 heterocephalus_glaber_RepeatModeler
503 Mar 12 01:01 heterocephalus_glaber_Database
119 Mar 12 03:18 heterocephalus_glaber_strainer
105 Mar 12 03:18 heterocephalus_glaber_Curated_Library
198 Mar 12 03:42 heterocephalus_glaber_RepeatMasker_Against_Custom_Library
28 Mar 12 03:42 heterocephalus_glaber_mergedRepeats
254 Mar 12 04:05 heterocephalus_glaber_RepeatLandscape
733 Mar 12 04:05 heterocephalus_glaber_summaryFiles
15482752 Mar 12 04:05 heterocephalus_glaberEarlGrey.log
```

As expected, everything needed for downstream genome annotation is in the “heterocephalus_glaber_summaryFiles” directory. The files needed for the rest of the pipeline are “heterocephalus_glaber.filteredRepeats.bed”, which contains the curated set of repeats, and “heterocephalus_glaber.softmasked.fasta”, which is the fasta file used in Braker and TOGA annotations. “heterocephalus_glaber.filteredRepeats.bed” is used in non-coding RNA seeding, as scRNA, srpRNA, and “SINE/tRNA-RTE” are called as part of repeat elements in earl grey. The other files are useful for summarizing and analyzing repeats.

The *species_summaryFiles* directory in EarlGrey

```
8025 Mar 12 03:43 heterocephalus_glaber.summaryPie.pdf
512 Mar 12 03:43 heterocephalus_glaber.highLevelCount.txt
46420 Mar 12 03:43 heterocephalus_glaber.familyLevelCount.txt
8201 Mar 12 04:05 heterocephalus_glaber_classification_landscape.pdf
9555 Mar 12 04:05 heterocephalus_glaber_split_class_landscape.pdf
13835 Mar 12 04:05 heterocephalus_glaber_superfamily_div_plot.pdf
100198 Mar 12 04:05 heterocephalus_glaber_divergence_summary_table.tsv
3847438 Mar 12 04:05 heterocephalus_glaber.filteredRepeats.bed
11296362 Mar 12 04:05 heterocephalus_glaber.filteredRepeats.gff
7582565 Mar 12 04:05 heterocephalus_glaber_combined_library.fasta
2236928 Mar 12 04:05 heterocephalus_glaber-families.fa.strained
44782565 Mar 12 04:05 heterocephalus_glaber.softmasked.fasta
```

EarlGrey 5.1.0 (the version at the time of GenAnT's release) recomputes TE divergence after finalizing TE libraries. This step is very time-consuming, and we have experienced instances on a slow node where this step alone can run for 200 hours. You do not need to wait for these divergences to be recalculated if you don't want to. If you navigate to the directory below, you can extract the "heterocephalus_glaber.filteredRepeats.bed" and then mask your assembly with `bedtools maskfasta`

...

```
ls
earl_grey/heterocephalus_glaber_EarlGrey/heterocephalus_glaber_mergedRepeats/looseMerge/
```

...

The *species_mergedRepeats/looseMerge* directory in EarlGrey where repeat annotations are stored.

heterocephalus_glaber_mergedRepeats/looseMerge/

```
180 Mar 12 03:42 heterocephalus_glaberLtrFinder
10455269 Mar 12 03:43 heterocephalus_glaber.out.gff3
11302980 Mar 12 03:43 heterocephalus_glaber.out.gff2.bak
11302980 Mar 12 03:43 heterocephalus_glaber.out.gff2
33 Mar 12 03:43 heterocephalus_glaberRepeatCraft
2813 Mar 12 03:43 ltrfinder_reformat_label.gff
11022968 Mar 12 03:43 heterocephalus_glaber.rclabel.gff
10064485 Mar 12 03:43 heterocephalus_glaber.rmerge.gff
2014 Mar 12 03:43 heterocephalus_glaber.summary.txt
10064485 Mar 12 03:43 heterocephalus_glaber.rmerge.gff.sorted
3847438 Mar 12 03:43 heterocephalus_glaber.mergedRepeats.bed
527 Mar 12 03:43 heterocephalus_glaber.mergedRepeats.revisedTable
527 Mar 12 03:43 heterocephalus_glaber.filteredRepeats.summary
3847438 Mar 12 03:43 heterocephalus_glaber.filteredRepeats.bed.bak
3847438 Mar 12 03:43 heterocephalus_glaber.filteredRepeats.bed
9965874 Mar 12 03:43 heterocephalus_glaber.rmerge.gff.filtered
11296362 Mar 12 03:57 heterocephalus_glaber.filteredRepeats.gff
```

Step 2: Generating protein-coding gene models

We use stringtie, braker, liftOff, and TOGA to call protein-coding and non-coding gene models in GenAnT.

Stringtie

GenAnT assumes that you have merged bam files at the tissue level. It will run stringtie for each tissue before merging them with stringtie merge. Stringtie is run with the “transcript_asm_stringtie” rule.

```
...
rule transcript_asm_stringtie:
    input:
        config["outDir"]+"/assembly/assembly.fa",
        config["outDir"]+"/RNAseq_alignment",
        config["outDir"]+"/ISOseq_alignment"
    output:
        config["outDir"]+"/transcript_selection/stringtie.gffread.gff"
...
```

Based on matching file names in “RNAseq_alignment” and “ISOseq_alignment” we first identify which tissues get stringtie-short read, stringtie-long read, and stringtie-mixed. These designations are stored in the text files in “stringtie_out”. After performing the correct version of stringtie. These annotations are combined into `stringtie.merged.gtf`. These annotations are then converted into a GFF file with “gffread” and moved into “transcript_selection”. StringTie does not require a softmasked genome and therefore uses “[assembly.fa](#)” to improve parallelization.

The “stringtie_out” directory

```
53 May 2 14:25 mixed.txt
138 May 2 14:25 rnaseq_only.txt
0 May 2 14:25 isoseq_only.txt
78064727 May 2 14:29 kidney.merged.bam.mix.gtf
67808715 May 2 14:34 liver.merged.bam.mix.gtf
108795526 May 2 14:38 testis.merged.bam.mix.gtf
54050033 May 2 14:39 adrenal.merged.bam.sr.gtf
70726597 May 2 14:42 cerebellum.merged.bam.sr.gtf
75672578 May 2 14:44 hypothalamus.merged.bam.sr.gtf
73890142 May 2 14:47 ovary.merged.bam.sr.gtf
60777222 May 2 14:48 pituitary.merged.bam.sr.gtf
57032336 May 2 14:50 skin.merged.bam.sr.gtf
55034897 May 2 14:51 thyroid.merged.bam.sr.gtf
96597221 May 2 14:52 stringtie.merged.gtf
```

Braker

Braker assumes that you have a softmasked assembly and RNA-seq data in the same format as stringtie. It does not have a “mix” option, and will therefore automatically know whether to run braker in short read mode and/or long read mode, or noRNA-seq mode, depending on whether there are bam files in the “RNAseq_alignment” and “ISOseq_alignment” directories. We have also added a “stranded” option in the config file to use short-read RNA-seq in Braker’s stranded vs. unstranded options. Our snakemake performs these tasks with the following rules below:

```
...
rule braker_noRNA:
    input:
        config["outDir"]+"/assembly/assembly.softmasked.fa",
        config["outDir"]+"/RNAseq_alignment",
        config["outDir"]+"/ISOseq_alignment"
    output:
        config["outDir"]+"/transcript_selection/braker.noRNA.gffread.gff"
...
...
    input:
        config["outDir"]+"/assembly/assembly.softmasked.fa",
        config["outDir"]+"/RNAseq_alignment",
        config["outDir"]+"/ISOseq_alignment"
    output:
        config["outDir"]+"/transcript_selection/braker.sr.gffread.gff"
...
...
rule braker_lr:
    input:
        config["outDir"]+"/assembly/assembly.softmasked.fa",
        config["outDir"]+"/RNAseq_alignment",
        config["outDir"]+"/ISOseq_alignment"
    output:
        config["outDir"]+"/transcript_selection/braker.lr.gffread.gff"
...
```

Each run of braker (assuming it is run) makes its directory, namely braker_lr, braker_sr, and braker_noRNA. Each directory contains the standard braker output.

The braker_sr directory. Braker_lr a braker_noRNA directories will look the same.

```
0 May 2 14:25 cache
0 May 2 14:25 tmp
2200 May 2 14:29 genome_header.map
125885980 May 3 21:25 hintsfile.gff
27 May 6 15:49 species
2530 May 8 01:42 what-to-cite.txt
100155313 May 8 01:56 braker.gtf
59151256 May 8 01:58 braker.codingseq
19964497 May 8 01:58 braker.aa
681 May 8 01:59 errors
254 May 8 02:05 GeneMark-ETP
113 May 8 02:05 Augustus
983325 May 8 02:05 braker.log
1137 May 8 02:05 brakerlog.log
```

The “braker.gtf” file is converted to a GFF file and moved to transcript_selection. The “braker.log” file should be used for potential debugging. In your run, if you see “cachedir” and “tempdir” instead of “cache” and “temp”, it’s to deal with a very niche bug fix when using braker noRNA-seq in Singularity.

LiftOff

LiftOff performs homology-based gene transfer of closely related species and is run with “liftoff_asm”. The output directory is very simple.

...

rule liftoff_asm:

input:

config["outDir"]+"/assembly/assembly.fa"

output:

config["outDir"]+"/transcript_selection/liftoff.gffread.gff"

...

The liftoff_asm directory

```
597225472 May 2 14:23 referencegff_db
89 May 2 14:29 intermediate_files
432 May 2 14:51 unmapped.liftoff.txt
129577938 May 2 15:14 liftoff.gff
```

We build the reference database in the “liftoff_asm” to use the reference genome for multiple assembly runs simultaneously. LiftOff will crash if multiple runs of the tool try to build the “_db” simultaneously. Unmapped.liftoff.txt reports the genes that could not be transferred between assemblies. liftoff.gff is what is converted to a GFFread-compatible GFF file and moved to transcript_selection.

TOGA

We have introduced using two TOGA runs as an option in GenAnT. The outputs are stored in “toga_out1”, which we usually use as a model organism for (e.g., mouse), and “toga_out2”, which works well for a moderately closely related species (e.g., >5Mya divergence and should perform better than LiftOff. TOGA is run with the “toga_anno”.

```
...  
rule toga_anno:  
  input:  
    config["outDir"]+"/assembly/assembly.softmasked.fa"  
  output:  
    config["outDir"]+"/transcript_selection/toga.gffread.gff"  
...
```

Each run of TOGA makes two output directories. The first is “cactus_alnr1”, which stores the pairwise alignment information between the reference and target species. The file used in the annotation is the “target-to-ref.chain” file.

The cactus output directory (cactus_out1)

```
317 May 2 14:22 cactus_in.txt  
26 May 4 15:47 toilwf-2b9fa31cf6ce5668b40bb8086fc765f4  
3014263898 May 4 15:47 target_ref.hal  
2762326063 May 4 15:50 mouse.fa  
755460557 May 4 15:51 mouse.2bit  
2661715239 May 4 15:51 Pup3.fa  
677552262 May 4 15:52 Pup3.2bit  
1319 May 4 15:52 mouse.bed  
2149 May 4 15:52 Pup3.bed  
994057403 May 4 16:25 mouse-to-Pup3.psl  
245983157 May 4 16:35 target-to-ref.chain
```

The directory made in TOGA is “toga_out1”, which contains the CESAR2.0 aligner binary and the output of TOGA. In this example, “Pup3” is the “target” name used in this assembly.

CESAR2.0 Pup3_toga

The output of TOGA has a lot of potentially useful files for comparative genomics analyses, with “query_annotation.gtf” being what is passed onto transcript_selection.

The TOGA output directory (toga_out1)

```
1774 May 4 16:36 project_args.json
 82 May 4 16:36 version.txt
6449239 May 4 16:42 orthology_scores.tsv
13324 May 4 16:43 proc_pseudogenes.bed
367240718 May 4 19:15 nucleotide.fasta
134779740 May 4 19:15 prot.fasta
514939578 May 4 19:16 codon.fasta
17012648 May 4 19:16 query_annotation.bed
 7111861 May 4 19:16 loss_summ_data.tsv
3301795 May 4 19:16 query_isoforms.tsv
 775211 May 4 19:16 query_gene_spans.bed
7034086 May 4 19:17 orthology_classification.tsv
129426 May 4 19:17 ref_orphan_transcripts.txt
 884961 May 4 19:17 genes_rejection_reason.tsv
110152272 May 4 19:17 inact_mut_data.txt
 921 May 4 19:17 temp
 35 May 4 19:17 done.status
191283030 May 4 19:17 toga_2025_05_04_at_16_36.log
24769298 May 4 19:17 query_annotation.genePred
731333236 May 4 19:17 query_annotation.gtf
.. ..
```

While quite detailed, the “inact_mut_data.txt” explains the intactness of each transferred gene or transcript (see example below).

```

GENE: rna-XM_036159153.1
# rna-XM_036159153.1 37 1 1 ATG ATG->ATG masked ATG_1
# rna-XM_036159153.1 37 3 55 ATG ATG->ATG masked ATG_2
# rna-XM_036159153.1 37 4 82 ATG ATG->ATG masked ATG_3
# rna-XM_036159153.1 37 8 214 ATG ATG->ATG masked ATG_4
# rna-XM_036159153.1 37 9 239 ATG ATG->ATG masked ATG_5
# rna-XM_036159153.1 37 11 290 ATG GTG->ATG masked ATG_6
# rna-XM_036159153.1 37 12 308 ATG ATG->ATG masked ATG_7
# rna-XM_036159153.1 37 13 341 ATG CTG->ATG masked ATG_8
# rna-XM_036159153.1 37 15 374 ATG ATG->ATG masked ATG_9
# rna-XM_036159153.1 37 15 398 ATG ATG->ATG masked ATG_10
# rna-XM_036159153.1 37 17 470 ATG ATG->ATG masked ATG_11
# rna-XM_036159153.1 37 18 488 ATG ATG->ATG masked ATG_12
# rna-XM_036159153.1 37 18 506 ATG ATG->ATG masked ATG_13
# rna-XM_036159153.1 37 19 511 ATG ATG->ATG masked ATG_14
# rna-XM_036159153.1 37 19 516 ATG ATG->ATG masked ATG_15
# rna-XM_036159153.1 37 19 542 ATG ATG->ATG masked ATG_16
# rna-XM_036159153.1 37 19 554 ATG ATG->ATG masked ATG_17
# rna-XM_036159153.1 37 20 590 ATG ATG->ATG masked ATG_18
# rna-XM_036159153.1 37 20 592 ATG ATG->ATG masked ATG_19
# rna-XM_036159153.1 37 21 635 ATG ATG->ATG masked ATG_20
# rna-XM_036159153.1 37 22 655 ATG ATG->ATG masked ATG_21
# rna-XM_036159153.1 37 22 682 ATG ATG->ATG masked ATG_22
# rna-XM_036159153.1 37 23 707 ATG ATG->ATG masked ATG_23
# rna-XM_036159153.1 37 23 718 ATG ATG->ATG masked ATG_24
# rna-XM_036159153.1 37 28 840 ATG ATG->ATG masked ATG_25
# rna-XM_036159153.1 37 28 855 ATG ATG->ATG masked ATG_26
# rna-XM_036159153.1 37 30 887 ATG ATG->ATG masked ATG_27
# rna-XM_036159153.1 37 30 903 ATG ATG->ATG masked ATG_28
# rna-XM_036159153.1 37 30 911 ATG ATG->ATG masked ATG_29
# rna-XM_036159153.1 37 30 922 ATG ATG->ATG masked ATG_30
# rna-XM_036159153.1 37 30 947 ATG ATG->ATG masked ATG_31
# rna-XM_036159153.1 37 INTACT_PERC_IGNORE_M 1.0
# rna-XM_036159153.1 37 INTACT_PERC_INTACT_M 1.0
# rna-XM_036159153.1 37 INTACT_CODONS_PROP 1.0
# rna-XM_036159153.1 37 OUT_OF_CHAIN_PROP 0.0
# rna-XM_036159153.1 37 MIDDLE_IS_INTACT TRUE
# rna-XM_036159153.1 37 MIDDLE_IS_PRESENT TRUE

```

The orthology_classification.tsv describes may also help with ambiguous gene identification orthology.

t_gene	t_transcript	q_gene	q_transcript	orthology_class
gene-Gm572	rna-NM_001085505.1	TOGA_00000006708	rna-NM_001085505.1.78	one2one
gene-Gm572	rna-XM_036164088.1	TOGA_00000006708	rna-XM_036164088.1.78	one2one
gene-Gm572	rna-NM_001368835.1	TOGA_00000006708	rna-NM_001368835.1.78	one2one
gene-Gm572	rna-XM_036164087.1	TOGA_00000006708	rna-XM_036164087.1.78	one2one
gene-Gm572	rna-XM_006538816.3	TOGA_00000006708	rna-XM_006538816.3.78	one2one
gene-Gm572	rna-XM_006538818.2	TOGA_00000006708	rna-XM_006538818.2.78	one2one
gene-Lrrfip2	rna-XM_030244587.2	TOGA_00000008277	rna-XM_030244587.2.98	one2one
gene-Lrrfip2	rna-XM_030244599.2	TOGA_00000008277	rna-XM_030244599.2.98	one2one
gene-Lrrfip2	rna-XM_030244597.1	TOGA_00000008277	rna-XM_030244597.1.98	one2one
gene-Lrrfip2	rna-XM_030244589.1	TOGA_00000008277	rna-XM_030244589.1.98	one2one
gene-Lrrfip2	rna-NM_001374664.1	TOGA_00000008277	rna-NM_001374664.1.98	one2one

Step 3: Combining and filtering gene models

Prepare transcripts

Step 3 is almost entirely completed in the “transcript_selection” directory. Preparing for transcript selection starts with the “mikado_prepare” rule. This rule waits until each possible genome annotation is generated. If there is no reason for the annotation to be generated (i.e., you have RNA-seq data and there is no need for the braker.noRNA.gffread.gff), then it will be copied in as an empty file.

```
...
rule mikado_prepare:
    input:
        config["outDir"]+"/assembly/assembly.softmasked.fa",
        config["outDir"]+"/transcript_selection/custom.gffread.gff",
        config["outDir"]+"/transcript_selection/stringtie.gffread.gff",
        config["outDir"]+"/transcript_selection/liftoff.gffread.gff",
        config["outDir"]+"/transcript_selection/braker.noRNA.gffread.gff",
        config["outDir"]+"/transcript_selection/braker.sr.gffread.gff",
        config["outDir"]+"/transcript_selection/braker.lr.gffread.gff",
        config["outDir"]+"/transcript_selection/togar1.gffread.gff",
        config["outDir"]+"/transcript_selection/togar2.gffread.gff",
    output:
        config["outDir"]+"/transcript_selection/mikado_prepared.fasta.fai"
...
```

The first thing we do is move the empty annotations, which in this case were braker_noRNAseq and a potential “custom” GFF file, into the “excluded_input” directory.

The transcript_selection directory at the end of step 2

```
58633510 May  2 14:53 stringtie.gffread.gff
88611885 May  2 15:14 liftoff.gffread.gff
58507785 May  4 19:18 toga.r1.faa
500328565 May  4 19:18 toga.r1.gffread.gff
45356997 May  8 10:43 braker.sr.gffread.gff
35899441 May  8 10:44 braker.lr.gffread.gff
115 May  8 10:47 excluded_input
```

I would recommend that, even after a successful run of GenAnT, to check the “excluded_input” directory to make sure that it has everything you’d expect. For example, if you have ISO-seq data but made a typo in the config file such that the ISO-seq bam files are not included in the analysis (e.g., you accidentally linked the fastq directory instead of the bam file directory), GenAnT will probably run but will include “[braker.lr.gffread.gff](#)” in excluded_input. In our GenAnT run, we only used one reference for TOGA, had RNA-seq and ISO-seq data, and did not use a custom GFF that was built for the same assembly. These file sizes are “0”.

The “excluded_input” directory, when we only used one reference to TOGA, had RNA-seq and ISO-seq, and did not include a custom GFF.

```
0 May  2 14:22 custom.gffread.gff
0 May  2 14:22 toga.r2.gffread.gff
0 May  8 10:46 braker.noRNA.gffread.gff
```

The same rule will preprocess the transcript fasta file and merge individual annotations for eventual transcript selection.

The mikado_prepare rule automatically continues until we run multiple transcript evaluation tools.

The transcript_selection directory after mikado prepare is finished

```
58633510 May  2 14:53 stringtie.gffread.gff
88611885 May  2 15:14 liftoff.gffread.gff
58507785 May  4 19:18 toga.r1.faa
500328565 May  4 19:18 toga.r1.gffread.gff
45356997 May  8 10:43 braker.sr.gffread.gff
35899441 May  8 10:44 braker.lr.gffread.gff
  115 May  8 10:47 excluded_input
   44 May  8 10:47 mikado
  911 May  8 10:47 mikado_input_sheet.txt
 7105 May  8 10:48 config.yaml
572216743 May  8 10:52 mikado_prepared.fasta
494383785 May  8 10:52 mikado_prepared.gtf
 20675458 May  8 10:52 prepare.log
 8847042 May  8 10:54 mikado_prepared.fasta.fai
```

“mikado_prepared.gtf”

```
chr1 stringtie transcript 31528 33101 . + . gene_id "stringtie_MSTRG.1.1.gene"; transcript_id "stringtie_MSTRG.1.1";
chr1 stringtie exon 31528 33101 . + . gene_id "stringtie_MSTRG.1.1.gene"; transcript_id "stringtie_MSTRG.1.1";
chr1 braker.lr mRNA 62600 67653 . - . gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1"; Name "braker.lr_g1.t1"; canonical_number "2"; canonical_proportion "1.0"; canonical_junctions "1,2"; has_start_codon "True"; has_stop_codon "True";
chr1 braker.lr CDS 62600 62776 . - 0 gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1";
chr1 braker.lr exon 62600 62776 . - . gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1";
chr1 braker.lr CDS 63219 63278 . - 0 gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1";
chr1 braker.lr exon 63219 63278 . - . gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1";
chr1 braker.lr CDS 67648 67653 . - 0 gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1";
chr1 braker.lr exon 67648 67653 . - . gene_id "braker.lr_g1.t1.gene"; transcript_id "braker.lr_g1.t1";
chr1 stringtie transcript 69710 70061 . . . gene_id "stringtie_MSTRG.2.1.gene"; transcript_id "stringtie_MSTRG.2.1";
```

“mikado_prepared.fasta”

```
>stringtie_MSTRG.1.1
aaaaaatctcacatatgcagatagttacaattggggatctgctacttggtcacaggcccaa
gaatgctgtcaaattatattcacctaaggttaattatgtctaataagaccgaatgtgctgt
atacagaaattgtacaactaaatgtctgcaagttaaacgtttggaacatgtcctgccg
tggtgaacaagaaattatgtatagtaattgccatgtctctttgccaatgggttggtttta
cacttgtaagggaaacttttaattacattccggccaacttgagtgacactcgttggtg
cttgagtcggttggtctgtgatattaccagacagaactgatttgattgtacctaatacgaat
tagacgggtggtcgccgtataccgccccgggatgtaatgatgatgtgggtctattgactga
tgcagaagtgttagtcgataggggtctttgttgctctgccgtccatcggggtgcacgc
ggaagcgggcagtagcaagattagcctgtaaagtggtaaaaggattgaatgccacctccac
```

Evaluate transcripts

Evaluating transcripts involves open-reading frame finding with “transdecoder”, querying predicted protein sequences against the UniProt database with “blastp”, and finding splice junctions with “Portcullis” for short read RNA-seq and “Regtools” for long-read RNAseq.

Transdecoder

ORF-finding is generated with the “transdecoder” rule

```
...
rule transdecoder:
    input:
        config["outDir"]+"/transcript_selection/mikado_prepared.fasta.fai"
    output:

        config["outDir"]+"/transcript_selection/transdecoder/transdecoder/mikado_prepared.fasta
        .transdecoder.bed"
...
```

The main output of transdecoder that we use is the “bed” file, which designates ORFs that are evaluated in Mikado.

The transdecoder directory in transcript_selection

```
1888 May 8 12:38 mikado_prepared.fasta.transdecoder_dir
184126473 May 8 12:40 mikado_prepared.fasta.transdecoder.gff3
147104476 May 8 12:44 mikado_prepared.fasta.transdecoder.pep
362414051 May 8 12:47 mikado_prepared.fasta.transdecoder.cds
41993610 May 8 12:48 mikado_prepared.fasta.transdecoder.bed
```

Portcullis and regtools

Finding splice junctions is generated with the “junctions” tool.

```
...
rule junctions:
    input:
        config["outDir"]+"/transcript_selection/mikado_prepared.fasta.fai"
    output:
        config["outDir"]+"/transcript_selection/junctions.final.bed"
    threads:
        16
    params:
        memory_per_thread="12G",
        extra_cluster_opt="",
        walltime="48:00:00"
    shell:
        """
        scripts/get_junctions.sh {config[outDir]}
        """
...
```

...

The output of finding junctions is “lr_junctions.bed” (with RegTools) and the output of Portcullis in transcript_selection/3-filt

The 3-filt directory (output of Portcullis) in transcript_selection

```
42 May 9 00:51 portcullis_filtered.selftrain.initialset.L95_intron_size.txt
58174345 May 9 00:51 portcullis_filtered.selftrain.initialset.pos.junctions.tab
264313 May 9 00:52 portcullis_filtered.selftrain.initialset.neg.junctions.tab
8094617 May 9 00:56 portcullis_filtered.selftrain.forest
219562115 May 9 00:58 portcullis_filtered.pass.junctions.tab
87979545 May 9 00:58 portcullis_filtered.pass.junctions.bed
```

Blastp

Blastp is executed with two rules, “split_fasta” and “blastp_run”. Split fasta separates the predicted protein fasta file into 100 sections, to improve runtime and parallelization.

...

```
rule split_fasta:
  input:
    config["outDir"]+"/transcript_selection/mikado_prepared.fasta.fai"
  output:
    expand("{outDir}/transcript_selection/blast/output_{split_id}.fasta", split_id =
range(1, 100), outDir=config["outDir"])
```

...

The “blastp_run” rule is executed 100 times (or how many times you split the fasta)

...

```
rule blastp_run:
  input:
    config["outDir"]+"/transcript_selection/blast/output_{split_id}.fasta"
  output:
    config["outDir"]+"/transcript_selection/blast/output_{split_id}/blast_done.txt"
...
```

The output of the “blastp_run” rule lives in the “blast” directory in transcript_selection. Each output (e.g., output_001 directory) is the result of a different blast run, which gets combined in blast_results.tsv

The blast directory in transcript_selection

```
blast_results.tsv  output_30  output_73  output_91  output_49  output_21  output_97.fasta  output_80.fasta  output_63.fasta  output_46.fasta  output_29.fasta  output_12.fasta
output_28         output_40  output_71  output_58  output_78  output_86  output_96.fasta  output_79.fasta  output_62.fasta  output_45.fasta  output_28.fasta  output_11.fasta
output_50         output_48  output_36  output_13  output_47  output_97  output_95.fasta  output_78.fasta  output_61.fasta  output_44.fasta  output_27.fasta  output_10.fasta
output_10        output_37  output_9   output_17  output_38  output_79  output_94.fasta  output_77.fasta  output_68.fasta  output_43.fasta  output_26.fasta  output_9.fasta
output_5         output_41  output_54  output_44  output_6   output_34  output_93.fasta  output_76.fasta  output_59.fasta  output_42.fasta  output_25.fasta  output_8.fasta
output_55        output_22  output_66  output_88  output_95  output_96  output_92.fasta  output_75.fasta  output_58.fasta  output_41.fasta  output_24.fasta  output_7.fasta
output_31        output_90  output_23  output_32  output_45  output_11  output_91.fasta  output_74.fasta  output_57.fasta  output_40.fasta  output_23.fasta  output_6.fasta
output_27        output_15  output_28  output_69  output_16  output_76  output_90.fasta  output_73.fasta  output_56.fasta  output_39.fasta  output_22.fasta  output_5.fasta
output_2         output_8   output_52  output_80  output_7   output_63  output_89.fasta  output_72.fasta  output_55.fasta  output_38.fasta  output_21.fasta  output_4.fasta
output_12        output_92  output_46  output_58  output_29  output_74  output_88.fasta  output_71.fasta  output_54.fasta  output_37.fasta  output_20.fasta  output_3.fasta
output_60        output_98  output_62  output_18  output_25  output_61  output_87.fasta  output_70.fasta  output_53.fasta  output_36.fasta  output_19.fasta  output_2.fasta
output_59        output_26  output_39  output_43  output_4   output_35  output_86.fasta  output_69.fasta  output_52.fasta  output_35.fasta  output_18.fasta  output_1.fasta
output_67        output_64  output_65  output_70  output_56  output_81  output_85.fasta  output_68.fasta  output_51.fasta  output_34.fasta  output_17.fasta  output_100.fasta
output_89        output_53  output_75  output_42  output_82  output_19  output_84.fasta  output_67.fasta  output_50.fasta  output_33.fasta  output_16.fasta
output_87        output_93  output_14  output_83  output_72  output_99  output_83.fasta  output_66.fasta  output_49.fasta  output_32.fasta  output_15.fasta
output_33        output_94  output_24  output_85  output_3   output_99.fasta  output_82.fasta  output_65.fasta  output_48.fasta  output_31.fasta  output_14.fasta
output_77        output_84  output_1   output_51  output_57  output_98.fasta  output_81.fasta  output_64.fasta  output_47.fasta  output_30.fasta  output_13.fasta
```

Transcript selection.

With each transcript evaluation step complete, we generate consensus transcripts with the “serialize_pick” rule, which organizes results and runs the “mikado serialize” and “mikado pick” tools.

```
...
rule serialize_pick:
    input:
        config["outDir"]+"/transcript_selection/mikado_prepared.fasta.fai",
        expand("{outDir}/transcript_selection/blast/output_{split_id}/blast_done.txt", split_id
= range(1, 100), outDir=config["outDir"]),
        config["outDir"]+"/transcript_selection/junctions.final.bed",

    config["outDir"]+"/transcript_selection/transdecoder/transdecoder/mikado_prepared.fasta
.transdecoder.bed"
    output:
        config["outDir"]+"/transcript_selection/mikado_lenient.faa"
...
```

The final transcript_selection directory will have all of the outputs from step 3. The final GFF file used in the next steps is “mikado_lenient.gff”. The “mikado_lenient.metrics.tsv” and “mikado_lenient.scores.tsv” are also very valuable, as they summarize metrics for every gene model (e.g., number of exons, Blast hits, ORF length, number of junctions, gene length ,etc.). Below is a screenshot of what the transcript_selection directory typically contains.

The transcript_selection directory at the end of step 3

```

58633510 May  2 14:53 stringtie.gffread.gff
88611885 May  2 15:14 liftoff.gffread.gff
58507785 May  4 19:18 toga.r1.faa
500328565 May  4 19:18 toga.r1.gffread.gff
45356997 May  8 10:43 braker.sr.gffread.gff
35899441 May  8 10:44 braker.lr.gffread.gff
   115 May  8 10:47 excluded_input
   44 May  8 10:47 mikado
  911 May  8 10:47 mikado_input_sheet.txt
 7105 May  8 10:48 config.yaml
572216743 May  8 10:52 mikado_prepared.fasta
494383785 May  8 10:52 mikado_prepared.gtf
 20675458 May  8 10:52 prepare.log
 8847042 May  8 10:54 mikado_prepared.fasta.fai
   30 May  8 10:54 transdecoder
47672674911 May  8 23:59 merged.sr.bam
 9364056 May  9 00:38 merged.sr.bam.bai
   92 May  9 00:50 2-junc
 396 May  9 00:58 3-filt
   0 May  9 00:58 1-prep
18418966695 May  9 02:04 merged.lr.bam
 16453736 May  9 02:09 merged.lr.bam.bai
 82984176 May  9 02:36 lr_junctions.bed
170963721 May  9 02:36 junctions.final.us.bed
170963656 May  9 02:37 junctions.final.bed
 5991 May  9 02:38 blast
1035378688 May  9 02:55 mikado.db
 7806 May  9 02:55 mikado_serialize.log
 21227363 May  9 05:52 mikado_lenient.metrics.tsv
 9425023 May  9 05:52 mikado_lenient.scores.tsv
168703751 May  9 05:52 mikado_lenient.gff
17852425 May  9 05:52 mikado_pick.log
34660459 May  9 05:56 mikado_lenient.faa

```

Step 4: Annotating non-coding RNA genes

MiRmachine

At this stage, protein-coding and lncRNAs are annotated in steps 2-3, however, these tools do not annotate short non-coding genes (rRNA, miRNA, etc.). We call miRNAs early on using mirmachine, as all we need is a softmasked assembly. We do this with the mirmachine rule.

```
...
rule mirmachine:
    input:
        config["outDir"]+"/assembly/assembly.softmasked.fa"
    output:
        config["outDir"]+"/mirmachine/mirna.filtered.gff"
...
```

The mirmachine output directory

```
48 May  9 05:58 analyses
29 May  9 07:42 results
23 May  9 07:42 data
82792 May  9 07:43 mirna.filtered.gff
```

Seeding and evaluation

We next use BLAST against the Rfam database and cmscan through Infernal to evaluate genome-wide small non-coding RNAs. The output of this stage is in “assembly.tblout”, and is in the “ncRNA_analysis” directory.

```
...
rule ncRNA_covariance:
    input:
        config["outDir"]+"/assembly/assembly.softmasked.fa",
        config["outDir"]+"/transcript_selection/mikado_lenient.faa"
    output:
        config["outDir"]+"/ncRNA_analysis/assembly.tblout"
...
```

The ncRNA_analysis directory after infernal successfully ran

```
5459972 May 9 06:31 assembly.rfam.blastn
3257161 May 9 06:31 assembly.rfam.bed
1285142 May 9 06:31 mikado_annotation.ncRNA.gff
317127 May 9 06:31 mikado_annotation.ncRNA.bed
0 May 9 06:31 liftoff_annotation.ncRNA.gff
0 May 9 06:31 custom_annotation.ncRNA.gff
14719 May 9 06:31 stringtie_annotation.ncRNA.gtf
6138 May 9 06:31 stringtie_annotation.ncRNA.bed
234496 May 9 06:32 earlGrey_annotation.ncRNA.bed
3814922 May 9 06:32 assembly_ncRNA_seed.bed
3814922 May 9 06:32 assembly_ncRNA_seed.s.bed
409650 May 9 06:32 assembly_ncRNA_seed.m.bed
55853474 May 9 06:32 assembly_ncRNA_seed.m.fasta
140515775 May 9 09:28 assembly.cmscan
430189 May 9 09:29 assembly.tblout
```

Incorporation into the GFF annotation file

After ncRNA is called, we include them into the GFF file. We do not worry about overlapping transcripts between coding and non-coding genes, as internal ncRNA genes exist. However, we do filter and polish short noncoding RNAs. These filtering and incorporation steps are generated with the “full_annotation” rule.

rule full_annotation:

input:

```
config["outDir"]+"/mirmachine/mirna.filtered.gff",
config["outDir"]+"/ncRNA_analysis/assembly.tblout"
```

The final “ncRNA_analysis” looks as below. The “full_annotation.gff” stores the final gene models generated in GenAnT but lacks gene symbols.

The ncRNA_analysis directory

5459972	May	9	06:31	assembly.rfam.blastn
3257161	May	9	06:31	assembly.rfam.bed
1285142	May	9	06:31	mikado_annotation.ncRNA.gff
317127	May	9	06:31	mikado_annotation.ncRNA.bed
0	May	9	06:31	liftoff_annotation.ncRNA.gff
0	May	9	06:31	custom_annotation.ncRNA.gff
14719	May	9	06:31	stringtie_annotation.ncRNA.gtf
6138	May	9	06:31	stringtie_annotation.ncRNA.bed
234496	May	9	06:32	earlGrey_annotation.ncRNA.bed
3814922	May	9	06:32	assembly_ncRNA_seed.bed
3814922	May	9	06:32	assembly_ncRNA_seed.s.bed
409650	May	9	06:32	assembly_ncRNA_seed.m.bed
55853474	May	9	06:32	assembly_ncRNA_seed.m.fasta
140515775	May	9	09:28	assembly.cmscan
430189	May	9	09:29	assembly.tblout
126487	May	9	09:30	infernal.gff
2019282	May	9	09:30	family.txt
236944	May	9	09:32	infernal.types.gff
8896	May	9	09:32	infernal.types.lncRNA.gff
1729	May	9	09:32	infernal.lncRNA.notInMikado.gff
168705480	May	9	09:32	mikado.infernal.gff
76246	May	9	09:32	mikado.infernal.lncRNALabeled.txt
41712	May	9	09:32	mikado.infernal.lncRNALabeled.mikadoInfo.gff
33723	May	9	09:32	mikado.infernal.lncRNALabeled.infernalInfo.gff
39286	May	9	09:32	mikado.infernal.lncRNALabeled.polished.gff
168404034	May	9	09:32	mikado.noLnc.gff
168443320	May	9	09:32	mikado.lncLabeled.gff
64900143	May	9	09:33	mikado.lncLabeled.exons.gff
228048	May	9	09:33	infernal.types.noLncRNA.gff
82792	May	9	09:33	mirmachine.gff
36570	May	9	09:33	mirmachine.id.gff
222078	May	9	09:33	infernal.noLnc.noMir.gff
258648	May	9	09:33	short_ncRNAs.gff
221839	May	9	09:33	short_ncRNAs.noOverlap.gff
952612	May	9	09:33	short_ncRNAs.polished.gff
169395932	May	9	09:33	full_annotation.unsorted.gff
169395792	May	9	09:34	full_annotation.gff

Step 5: Sequence-similarity-based transfer of gene symbols

By default, we call gene symbols with the results of TOGA, LiftOff, and orthofinder.

Orthofinder

Orthofinder identifies one-to-one, one-to-many, and many-to-many protein-coding gene matches between several species. In the case of genome annotation and GenAnT, this is between the reference and target species (i.e., two species). Orthofinder is run with the “run orthofinder” rule, and starts after step 3 (as it doesn’t include non-coding genes).

```
...
rule run_orthofinder:
    input:
        config["outDir"]+"/transcript_selection/mikado_lenient.faa",
    output:
        config["outDir"]+"/orthofinder/orthofinder_protein.tsv"
...
```

We automatically build the input directory for OrthoFinder, and then run the tool.

The orthofinder input directory

GCF_000001635.27_GRCm39_genomic.table.txt	mikado_lenient.txt	protein_seqs
mikado_lenient.gff	orthofinder_protein.tsv	Pup3_orthofinder

Like Earl Grey, OrthoFinder contains an analysis suite. As a result, the “Pup3_orthofinder” directory contains many subdirectories. Like in TOGA, many of these directories contain helpful information for computing comparative genomics statistics and calculating orthologs. We use the specific ortholog mapping in the Orthologues directory. This file is the tsv file stored in “Orthologues/Orthologues_mikado_lenient”

The orthofinder directory after a complete run

```
1307010 May 9 07:20 Orthogroup_Sequences
698730 May 9 08:29 MultipleSequenceAlignments
  92 May 9 08:29 Species_Tree
  24 May 9 08:29 Phylogenetic_Hierarchical_Orthogroups
 785 May 9 08:29 WorkingDirectory
   0 May 9 08:29 Phylogenetically_Misplaced_Genes
 104 May 9 08:29 Putative_Xenologs
425520 May 9 08:29 Gene_Trees
425520 May 9 08:29 Resolved_Gene_Trees
  97 May 9 08:29 Gene_Duplication_Events
 213 May 9 08:29 Orthogroups
210766 May 9 08:30 Single_Copy_Orthologue_Sequences
  42 May 9 08:30 Comparative_Genomics_Statistics
 224 May 9 08:30 Orthologues
 2522 May 9 08:30 Citation.txt
  609 May 9 08:30 Log.txt
```

Compiling gene symbols

The final two steps of the GenAnT workflow are to generate the gene symbol from each method to the gene ID called with mikado before generating the final annotation file, “full_annotation.geneSymbols.gff”. We do this with the aggregate symbols rule.

```
...
rule aggregate_symbols:
  input:
    config["outDir"]+"/ncRNA_analysis/full_annotation.gff",
    config["outDir"]+"/orthofinder/orthofinder_protein.tsv"
  output:
    config["outDir"]+"/full_annotation.geneSymbols.gff"
...
```

All intermediary files, which are essentially overlapping the named transcript from TOGA and liftOff with the final gene models from Mikado, are found in the “gene_symbol” directory (see below).

The “gene_symbol” directory

```

0 May  9 09:34 toga.mRNA.lncRNA.gff
0 May  9 09:34 mikado.toga.mRNA.lncRNA.txt
0 May  9 09:34 toga_overlap.mikadoInfo.gff
0 May  9 09:34 toga_overlap.togaInfo.gff
169395792 May 11 15:43 full_annotation.gff
500328565 May 11 15:43 toga.r1.gffread.gff
88611885 May 11 15:43 liftoff.gffread.gff
0 May 11 15:43 toga.r2.gffread.gff
18169395 May 11 15:43 mikado.mRNA.lncRNA.gff
5393270 May 11 15:44 liftoff.mRNA.lncRNA.gff
23432960 May 11 15:44 toga.r1.mRNA.lncRNA.gff
0 May 11 15:44 toga.r2.mRNA.lncRNA.gff
87652669 May 11 15:44 mikado.liftoff.mRNA.lncRNA.txt
219459995 May 11 15:44 mikado.toga.r1.mRNA.lncRNA.txt
0 May 11 15:44 mikado.toga.r2.mRNA.lncRNA.txt
66235427 May 11 15:44 liftoff_overlap.mikadoInfo.gff
20320353 May 11 15:44 liftoff_overlap.liftoffInfo.gff
126500121 May 11 15:44 toga_overlap.r1.mikadoInfo.gff
90845297 May 11 15:44 toga_overlap.r1.togaInfo.gff
0 May 11 15:44 toga_overlap.r2.mikadoInfo.gff
0 May 11 15:44 toga_overlap.r2.togaInfo.gff
2439829 May 11 15:44 orthofinder_protein.tsv
2710444 May 11 15:44 reference.table.txt
2060869 May 11 15:59 gene_symbols.tsv
2010277 May 11 15:59 gene_symbols_noCopies.tsv
2629501 May 11 16:01 gene_symbols_full.tsv
```

The most important files in this directory that you may later need are the `gene_symbols.tsv` and `gene_symbols_noCopies.tsv` files. These are TSV files where rows are gene IDs and columns are the gene symbols called by each tool.

The `gene_symbols.tsv` table

mikado_id	ncRNA_gene	liftoff_gene	togar1_gene	orthofinder_gene
mikado.chr1G1	NA	NA	NA	NA
mikado.chr1G2	NA	NA	NA	NA
mikado.chr1G8	NA	NA	NA	NA
mikado.chr1G10	NA	NA	NA	NA
mikado.chr1G11	NA	RSPH4A	Rsph4a	Rsph4a
mikado.chr1G12	NA	ZUP1	Zup1	Zup1
mikado.chr1G13	NA	KPNA5	NA	NA
mikado.chr1G14	NA	KPNA5	NA	Mdm2
mikado.chr1G15	NA	NA	Fam162b	NA
mikado.chr1G16	NA	GPRC6A	Gprc6a	Gprc6a
mikado.chr1G17	NA	RFX6	Rfx6	Rfx6
mikado.chr1G18	NA	VGLL2	Vgll2	Vgll2